2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3503-2697-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/COMPSAC57700.2023.00283

Evaluating ChatGPT for Smart Contracts Vulnerability Correction

Emanuele Antonio Napoli
Department of Control and Computer Engineering
Politecnico di Torino
Turin, Italy
emanuele.napoli@polito.it

Valentina Gatteschi
Department of Control and Computer Engineering
Politecnico di Torino
Turin, Italy
valentina.gatteschi@polito.it

***Abstract*—The growing number of exploits and hacks on the Ethereum blockchain has led to the development of powerful smart contract vulnerability detection tools and the frequent patching of the smart contract's programming languages (such as Solidity). At the same time, an ever-increasing number of people are interested in blockchain and smart contract-related topics and willing to build and deploy their own Decentralized Applications (dApp). However, learning a new programming language and its best practices as long as how to actually deploy a smart contract on the blockchain is a difficult task even for experienced developers. Recently, ChatGPT, a new user-friendly deep learning tool, has been released to improve the ability of non-skilled users to write high-quality code and in general, to boost the performances of developers in key tasks related to code writing (i.e., writing functions, explaining runtime errors, fixing bugs, etc.). This paper aims to measure the capabilities of ChatGPT in fixing vulnerable smart contracts and to assess the effectiveness of this tool, determining whether it can be a valuable aid for those who want to correct their own smart contract or want to reuse existing ones by first checking their status and eventually fix their vulnerability. In particular, we asked ChatGPT to fix 143 smart contracts with well-known labeled vulnerabilities. We considered a vulnerability as "fixed" if the code corrected by ChatGPT no longer contained the vulnerability (for this purpose, we exploited Slither, one of the state-of-the-art tools for smart contracts vulnerability detection to check the status of the original and the corrected smart contracts). As a result we obtained that ChatGPT was able to fix bugs and vulnerable smart contracts on average the 57.1% of the time with an increase of +1.4% when a description of the bug was provided in addition to the smart contract's source code.**

***Index Terms*—blockchain, bug fix, ChatGPT, code correction, smart contracts, Solidity**

1. Introduction

Recently, an increasing number of technology enthusiasts have discovered the benefits of the blockchain, and become fascinated by this technology. A blockchain is a shared ledger, managed by network nodes on a peer-to-peer basis, which records transactions (i.e., messages) sent from one node to another. Since the same copy of the blockchain is stored on each node running the same protocol (i.e., Bitcoin, Ethereum, etc), anyone running a node can actually inspect the ledger without relying on any third party. Furthermore, data stored on the blockchain cannot be altered or erased. The first working implementation of blockchain was born to exchange value between individuals without relying on the services of an intermediary. Nonetheless, over time, this technology has evolved to allow the execution of pieces of code, the so-called smart contracts [1], autonomous and decentralized programs that trigger transactions when certain conditions are met. Given the above-mentioned characteristics, smart contracts have been used to develop applications based on various business logic (such as roulette, lotteries, wallets, crowdfunding, etc.) in order to reduce dependency on service providers (i.e., database providers, payment services, etc.) and thus reducing management costs. As a result, the number of deployed dApps (Decentralized Applications, applications based on smart contracts, executed in a fully autonomous way) drastically increased over time [2]. At the same time, the number of attacks on smart contracts, which hold a large number of tokens (which often have monetary value), has increased significantly. This is because the source code of smart contracts is publicly available and may contain vulnerabilities that can be exploited by hackers to steal funds. Usually, the vulnerabilities are due to the lack of experience of the developer, the misuse of an already-existing piece of code, or the immaturity of the programming language (i.e., Solidity) since the blockchain and smart contract technologies are still in their infancy.

Many smart contract vulnerability tools have been developed over the years both by academia and the community to assist developers coding smart contracts. However, these tools often require a local installation, in-deep command line knowledge as they do not have a Graphic User Interface (GUI), and they are only intended to be used in the final stages of the development to check for the presence of vulnerabilities in the source code or they are not intended to actually support the developer during the code writing phase.

Nowadays, artificial intelligence-based tools have become increasingly popular because of their ability to help

979-8-3503-2697-0/23/$31.00 ©2023 IEEE
DOI 10.1109/COMPSAC57700.2023.00283

Authorized licensed use limited to: Korea University. Downloaded on January 23,2026 at 08:37:30 UTC from IEEE Xplore. Restrictions apply.

non-skilled people to accomplish tasks that can be challenging even for a domain expert. In particular, deep learning-based sequence prediction models [3], [4] have been used in Natural Language Processing (NLP), computer vision, and speech processing. ChatGPT is an NLP-based artificial intelligence, released in November 2022, which gained massive popularity due to its ability to generate a conversational response to the users' questions. ChatGPT was initially trained as a common Large Language Model (LLM); then, under reinforcement learning settings, it learned from the ratings provided by humans on its response. Within a few months, the technology powering ChatGPT went from GPT-3 to GPT-3.5 to GPT-4. However, at the time of writing (April 2023), the latest version is only available to selected users and not for the wider public. ChatGPT could be used to perform complex tasks, such as writing or summarizing code or texts, or answering to complex questions.

In the literature, several research works already evaluated ChatGPT's performance in different domains, such as education [5], research [6], healthcare [7], or software development [8], to mention a few.

In this paper, we aim to test the capability of ChatGPT in fixing vulnerable and bugged Solidity code. Specifically, we asked ChatGPT to provide code correction in well-known vulnerable smart contracts. The contribution of the paper can be summarised as follows:

- evaluate ChatGPT's ability to correctly process requests related to the smart contract programming language;
- evaluate how often ChatGPT is able to fix the Solidity code provided;
- propose a framework (for the Ethereum blockchain) that leverages ChatGPT and static analysis in order to fix bugs and vulnerabilities in smart contracts.

The remainder of the paper is organised as follows: Section 2 presents some background on smart contracts. Section 3 provides an overview of recent research works analyzing the performance of ChatGPT in a variety of domains. Section 4 presents the approach that we devised to execute the requests to ChatGPT and evaluate its response. Section 5 discusses the results obtained. Finally, conclusions and future work are reported in Section 6.

2. Smart contracts

As previously mentioned, smart contracts enable the creation of distributed and decentralized applications, which execute business logic in an autonomous and transparent manner. Although several programming languages for the development of smart contracts exist, Solidity has emerged as the most popular. In particular, it enables the coding of smart contracts on the Ethereum ecosystem, the most popular blockchain supporting the execution of smart contracts.

In order to deploy a smart contract on the blockchain, the Solidity script must first be compiled by the Ethereum Virtual Machine (EVM) to obtain the so-called bytecode (i.e., lower-level instructions to be executed on the EVM). Then, a transaction containing the bytecode must be submitted to the Ethereum network for immutable storage. Often, a smart contract needs to interact with external applications (e.g., the dApps). In this case, together with the bytecode, its Application Binary Interface (ABI) must be provided. The ABI is a JSON file that contains information about the smart contract's functions, parameters, return values, etc.

After the smart contract is deployed, its functions can be triggered by sending a transaction to the smart contract's address. For each transaction, an additional amount of tokens must be provided, since the EVM requires a fee to be paid – the so-called "gas" – in order to perform operations. This behavior is designed to avoid the execution of infinite loops, which can freeze the network. Once the gas is exhausted, the execution is terminated and the state of the smart contract is reverted back.

Smart contract technology is still in its infancy, and bugs or vulnerabilities still continue to be found in existing smart contracts[1]. Despite the large number of vulnerabilities that have been discovered during the recent years [9], most smart contracts may still contain weaknesses. This may be due to the fact that they have not been thoroughly verified, or that they have been compiled and deployed with older versions of the Solidity language (more prone to bugs). Since documents deployed on the blockchain cannot be modified, old smart contracts cannot be patched. To address these issues, newer versions of Solidity have been released primarily to address security issues and compilation bugs.

Several vulnerability detection tools have been developed so far. The most popular are Slither [10], Manticore [11], Mythril [12], SmartCheck [13], Securify [14], and Oyente [15]. Slither, Manticore, and Mythril are the most updated ones, with a GitHub last commit dated 2023. SmartCheck, Securify, and Oyente are less up-to-date (the last commit is dated 2021 or earlier), but received a higher number of citations, according to Google Scholar (e.g., Oyente received more than 1800 citations, while Slither received more than 200 citations). Among all the tools Slither is the one that received the highest community rating, with 3.8K stars.

3. Related works

Recent literature reports several works aiming at evaluating the capabilities of ChatGPT in a wide range of applications. In fact, ChatGPT is expected to bring impressive changes in the way academic writing is performed, information is searched and retrieved, people write code or detect vulnerabilities, and in the creation/analysis of social media content [16].

ChatGPT has been extensively tested in the education sector, to verify whether it could replace humans/students in writing texts and reports [5]. In this context, some researchers were concerned about this tool [5], while others suggested to adjust the learning goals, focusing more on improving students' creativity and critical thinking, rather

1. https://hacken.io/insights/bonqdao-hack/

Authorized licensed use limited to: Korea University. Downloaded on January 23,2026 at 08:37:30 UTC from IEEE Xplore. Restrictions apply.

than performing tasks that could be carried out by the AI [17]. Other research works suggested that ChatGPT could be a valid tool for teachers, as it could simplify/speed-up the creation of educational content [18].

In the context of scientific writing, the capabilities of ChatGPT have been evaluated in tasks such as drafting a literature review [19], or writing a complete paper (performing idea generation, literature review, data identification/processing, and empirical testing) [6].

In the medical field, ChatGPT has been exploited to create reports from scratch [20] or simplify medical reports in order to make them understandable to non-educated patients [7]. Some works, such as [21], evaluated the ability of this tool to pass exams such as the United States Medical Licensing Examination.

In the area of software testing, researchers compared ChatGPT's bug-fixing capabilities with other deep learning and program repair techniques on existing datasets (such as the QuixBugs benchmark set [22]). The results showed that ChatGPT was correct only 47.5% of the time, results which are consistent with those of [23], where the AI tool fixed the 50% of code snippets. The authors of [22] also pointed out that a further hint as to what could be the problem/bug in the code could increase the success rate up to 77.5%. The work described in [24] took a step further, and proposed a conversational automated program repair tool based on the OpenAI `code-davinci-002` engine whose goal was to both generate code patches and validate them iteratively with user feedback. This solution avoided the generation of previously erroneous patches since the framework leveraged the conversation context window. A key advantage of ChatGPT, as the authors stated, relied on its dialog-like interaction with the user, who was able to provide additional information iteratively. The authors of [25] developed InferFix, an LLM-based program repair framework relying on a state-of-the-art static analyzer to correct security and performance issues and bugs. In particular, they tested program repair baselines that leveraged the OpenAI's `code-cushman-001` and `text-davinci-003` engines. Results showed that the latter was able to achieve up to 40-53% accuracy in fixing bugs. Moreover, the authors tried several techniques to improve performance such as stating that there was actually a bug in the prompted code, adding location information, providing extended context, and enriching context with hints reaching between 57% and 82% for three bug categories related to Java and C# programming languages.

Regarding the various models tested in recent research works, it should be noted that OpenAI models are constantly being updated and improved (i.e., `code-davinci-003` and `code-cushman-001` are now deprecated). At the time of this writing (April 2023) the most powerful model of the engines available to the general public is the `gpt-3.5-turbo` engine.

As it is possible to notice from the above overview of the recent literature, the opportunities, capabilities, and threats of ChatGPT in bug-fixing and software testing have been analyzed by only a few research papers since this tool has only recently been released and is continuously improving its performance. Therefore, the capabilities of this AI tool for tasks such as code writing, debugging, and testing still need to be thoroughly tested. Furthermore, to the best of our knowledge, no other works exist, which evaluate the capabilities of ChatGPT to correct vulnerable smart contracts. We aim to fill this gap by evaluating ChatGPT on a well-known dataset of vulnerable smart contracts, and by testing the corrected code generated by ChatGPT with an existing state-of-the-art smart contract vulnerability detection tool.

4. Proposed approach

In this Section describes the approach we followed to evaluate ChatGPT's capabilities in fixing smart contracts vulnerabilities. In particular, we relied on the SmartBug-curated dataset[2] for this evaluation. The SmartBug-curated is a dataset consisting of 143 vulnerable smart contracts, whose vulnerabilities have been manually annotated according to a well-known classification (i.e., the DASP classification[3]) [26]. It is worth noting that for the purposes of our study, we did not consider manual annotation, since the objective of this work was not to determine to what extent ChatGPT was able to identify vulnerabilities, but rather it was to evaluate the capability of this AI in the process of correcting a vulnerability in the code.

Therefore, based on the workflow of [25], we adopted the pipeline described in the following, which relies on Slither [10] in two different phases: a) before submitting a request to ChatGPT, in order to provide the AI with some hints about the contained vulnerability, and b) after receiving the corrected code from the AI, in order to verify the quality of the correction. As reported in Section 2, Slither is a well-known static analysis tool for vulnerability detection, which is among the most appreciated publicly available tools, both by academia and the community (i.e., in terms of received stars and citations). Furthermore, this tool is constantly updated and improved (whereas other publicly available tools are no longer maintained). In addition, other tools such as Manticore and Mythril – also well-appreciated by the community – are not competitive in terms of time resources, since they require a huge amount of time to navigate through all possible paths in order to double-check the presence of vulnerabilities. On the contrary, Slither can perform a vulnerability analysis in much less time (i.e., about 5 seconds).

In order to evaluate the quality of ChatGPT's response to smart contracts bug fixing, we developed the pipeline shown in Figure 1. The pipeline consists of four steps (depicted with orange rectangles).

The first step, *code cleaner*, is devoted to preprocessing the smart contract, in order to clean its source code from all the comments before analyzing it and submitting the request to ChatGPT. This step is carried out to minimize the number of tokens included in the question prompted to ChatGPT, due to the imposed limitation of 4096 tokens[4]. To perform

2. https://github.com/smartbugs/smartbugs-curated
3. https://dasp.co/
4. https://platform.openai.com/docs/models/gpt-3-5

Authorized licensed use limited to: Korea University. Downloaded on January 23,2026 at 08:37:30 UTC from IEEE Xplore. Restrictions apply.

this step, we applied a regex pattern removing all Solidity comments. Indeed, Solidity comments are expressed in two ways: a) starting a comment string with "//" (one-line comment) and b) including a multi-lines comment block inside the tags "/*...*/".

After this step, the cleaned smart contract is passed to the *vulnerability finder*, a module that leverages Slither in order to obtain both the location and the description of the potential threats discovered. This step is designed with two objectives: the first one is to identify the number of vulnerabilities prior to ChatGPT correction, in order to quantify how many vulnerabilities have been corrected by the AI, in the subsequent phases. The second objective is to provide ChatGPT with some indications about the type of vulnerability found by a state-of-the-art tool in the code. It is worth noting that, in the designed pipeline, the user could decide to ignore Slither's indications, and let ChatGPT alone infer the type of error and fix the code. During this step, the user could also optionally choose the severity level of the vulnerability that should be communicated to ChatGPT, in the following phase. In fact, since Slither also labels vulnerabilities according to their severity, the user could decide to focus only on certain vulnerabilities. If no level is specified, only high severity vulnerabilities will be considered by default.

The output of the *vulnerability finder* (or, in case this step is omitted, of the *code cleaner*) is then used to create a question to submit to ChatGPT. The question varies depending on whether the output of Slither is used or not. Basically, if Slither is not used, only the source code is considered, otherwise, the vulnerability description provided by Slither is also included in the question according to the following question schema.

Question without bug description provided by Slither

> Fix the vulnerability in the Solidity code. Provide the fixed contract. "<source code>"

Question with bug description provided by Slither

> Fix the vulnerability in the Solidity code. Provide the fixed contract. "<source code>"
> Errors:
> <error 1>
> <error 2>
> ...
> <error n>

The third step is the *ChatGPT questioner*. In this step, we used the OpenAI API[5] in order to get the ChatGPT response. In particular, we exploited the `gpt-3.5-turbo` model, the most powerful among the OpenAI's engines available to the wider public. To perform the requests, we added an offset of 1 second between calls in order to avoid

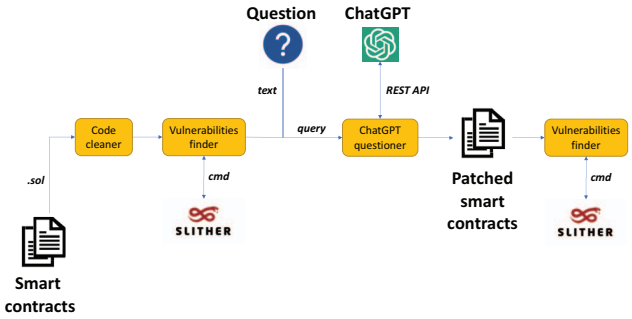


Figure 1. Designed pipeline

the 429 error[6] and overload the OpenAI servers. In case a call failed, the question was submitted again until a valid response was returned. For this reason, a skipping logic was implemented since the call could fail due to server overload or because the smart contract text was longer than the aforementioned limit of 4096 tokens. Hence, after a trial and error session, we set the number of times that a call could fail before the current smart contract was skipped to five, since it was assumed to exceed the current limits. The returned (supposedly) fixed code was then saved in a folder named as the original smart contract for further inspection (i.e., to double-check if vulnerabilities and errors are still present).

The last step exploits again the *vulnerability finder* module. This step is mandatory and is necessary to identify whether the corrected/patched smart contract's code is actually free from vulnerabilities, or, more in general, if the vulnerabilities submitted to ChatGPT for correction have been fixed or ChatGPT has actually added some bugs in the smart contract.

In order to investigate ChatGPT's ability to correct smart contract code, we exploited the pipeline just described as follows: we submitted both questions to ChatGPT with a) the source code alone and b) the source code followed by the bug description provided by Slither. The objective was to identify whether the results provided by ChatGPT could vary in case a state-of-the-art tool is used to provide the AI with some hints about the existing vulnerabilities. Following the approach of [22], each question was asked to ChatGPT four times, and we considered the best of the four results provided by the AI. Specifically, we considered a smart contract code as corrected if Slither detected fewer vulnerabilities (without introducing new ones) than those found in the original smart contract source code, at least in one over the four attempts (more details are provided in Section 5).

5. Results

This Section aims to present the results obtained by ChatGPT by a) providing ChatGPT with the source code

5. https://platform.openai.com/

6. https://platform.openai.com/docs/guides/error-codes/api-errors

Authorized licensed use limited to: Korea University. Downloaded on January 23,2026 at 08:37:30 UTC from IEEE Xplore. Restrictions apply.

TABLE 1. OVERALL RESULTS OBTAINED IN "CODE ONLY" AND "BUG DESCRIPTION" MODES.

	Code only	Bug description
Average	52.8%	53.1%
Best round	57.1%	58.5%
All bugs fixed	27.1%	26.7%

alone and b) also providing the description of the vulnerability detected by Slither. Detailed results for each smart contract/run along with the code developed for the analysis can be found on Github[7].

Table 1 shows the results obtained using only the source code and the Slither's vulnerability description, whereas Table 2 and Table 3 show the results for each round, and the severity of the vulnerabilities. In particular, the Tables 2 and 3 show how many vulnerabilities were fixed in each round. The Tables contain also the evaluation for the "best round" (i.e., the best round out of the four attempts). In particular, we selected the best round according the following logic:

- in a round, the new Solidity code must not contain any new vulnerabilities introduced by ChatGPT in order to proceed;
- starting from high severity vulnerabilities, we considered the round valid if ChatGPT fixed at least one vulnerability (taking as reference the vulnerabilities detected in the original source code) starting from the highest severity level to the lowest (i.e., "High", "Medium", "Low"); if the number of vulnerabilities of the same severity in a round is equal to the one detected in the original source code then we move to compare the number of vulnerabilities of lower severity level;
- if the round is valid, we counted the number of the fixed vulnerabilities for all severity levels;
- for each smart contract analysis, we selected as the best round the round with the largest number of fixed vulnerabilities among all the valid rounds;

In addition to the results reported in the Tables, it is worth mentioning that, as far as the results in "Code only" mode are concerned, the 2% (i.e., 3 out of 143) of all smart contract's source codes exceeded the aforementioned ChatGPT's current limit of 4096 tokens when embedded in the question. For this reason, ChatGPT did not provide any fixed code and the results were carried out considering only the 140 smart contracts of the dataset that were correctly processed by ChatGPT (hence, 560 requests received a response).

Furthermore, considering all the four rounds for all the smart contracts (i.e., the 560 requests), 56 ChatGPT responses provided a smart contract that Slither was not able to compile (hence, we can assume that the resulting code was incorrect).

The results in Table 1 show that ChatGPT was able to actually fix the code 52.8% of the time (i.e., 296 out of 560). In addition, ChatGPT was able to fix all the vulnerabilities detected by Slither in the source code 27.1% of the times (i.e., 38 out of 140). For this latter value, a smart contract is considered fully corrected if the Solidity code returned by ChatGPT has no errors (i.e., compilation errors) or vulnerabilities detected by Slither at least one round out of the four. Finally, taking into account the best round, ChatGPT was able to fix at least one vulnerability the 57.1% of the time (i.e., 78 out of 140).

Regarding the results obtained by providing ChatGPT with a description of the errors detected by Slither, 57 returned smart contracts have compilation issues. In addition, 8 out of 143 smart contracts, when embedded in the prompted question along with bugs description, exceeded the ChatGPT's token limit and did not return a fixed smart contract. Hence, the following results are based on 135 smart contracts correctly processed by ChatGPT. The 53.1% of all returned smart contracts by ChatGPT (i.e., 287 out of 540) had at least a vulnerability fixed by the AI, whereas ChatGPT was able to fix a vulnerability in the 58.5% of the processed smart contracts (i.e., 79 out of 135) picking the best round. In addition, ChatGPT was able to fix all bugs in at least one round in the 26.7% of the cases (i.e., 36 out of 135). This improvement in performance could be due to the fact that ChatGPT can infer more context from the error description, rather than from the source code only.

Concerning results reported in Tables 2 and 3, we can note that, on average, ChatGPT fixes a higher percentage of vulnerabilities – considering questions that did not exceed the number of tokens (140 smart contracts in "code only" and 135 "with bug description") – when a description of the error is added to the prompted question. In addition, ChatGPT returns on average a Solidity code with a lower number of vulnerabilities actually added by ChatGPT itself when the bug description is provided.

6. Conclusion

This work aimed to investigate the ability of ChatGPT to fix vulnerable smart contracts. To accomplish this task, we built a pipeline leveraging a state-of-the-art smart contract analysis tool, in order to evaluate both the original and the fixed smart contracts provided by ChatGPT. As a result, the evaluation showed that:

- ChatGPT was able to produce a compilable smart contract 89.8% of the time;
- ChatGPT was able to patch at least one vulnerability in 57.1% of the cases with an average of 4 attempts, and improved its performance by +1.4% when a description of the error was provided (assuming that the request did not exceed ChatGPT's token limit);

The pipeline we built, that leverages ChatGPT and Slither is available on Github, and can be used by developers to automatically fix bugs and vulnerabilities in smart contracts. As future work, we aim to enhance the pipeline with various smart contract analysis tools to further improve both vulnerability detection and ChatGPT response In addition,

7. https://github.com/enaples/solgpt

Authorized licensed use limited to: Korea University. Downloaded on January 23,2026 at 08:37:30 UTC from IEEE Xplore. Restrictions apply.

TABLE 2. FIXED AND ADDED VULNERABILITIES IN "CODE ONLY" MODE OVER 140 PROCESSED SMART CONTRACTS

	Source code	Round 1		Round 2		Round 3		Round 4		Best round	
Severity	**Vulnerability**	**Fixed**	**Added**	**Fixed**	**Added**	**Fixed**	**Added**	**Fixed**	**Added**	**Fixed**	**Added**
High	130	61 (44%)	9	61 (47%)	8	62 (47%)	7	62 (48%)	6	64 (49%)	6
Medium	115	77 (67%)	12	78 (68%)	11	79 (69%)	9	79 (69%)	10	81 (70%)	9
Low	169	69 (41%)	5	69 (41%)	4	73 (43%)	5	73 (43%)	4	76 (45%)	5

TABLE 3. FIXED AND ADDED VULNERABILITIES IN "BUG DESCRIPTION" MODE OVER 135 PROCESSED SMART CONTRACTS

	Source code	Round 1		Round 2		Round 3		Round 4		Best round	
Severity	**Vulnerability**	**Fixed**	**Added**	**Fixed**	**Added**	**Fixed**	**Added**	**Fixed**	**Added**	**Fixed**	**Added**
High	125	57 (46%)	9	57 (46%)	7	59 (47%)	7	59 (47%)	6	62 (49%)	5
Medium	111	76 (68%)	11	75 (68%)	10	78 (70%)	8	78 (70%)	10	80 (72%)	8
Low	162	72 (44%)	4	67 (41%)	3	78 (48%)	4	77 (47%)	3	82 (50%)	4

we aim to support the analysis of smart contracts related to other blockchains (i.e., EOS, Solana, etc.) and compare more LLMs along with ChatGPT. Moreover, we also aim to develop a tool that can be integrated into common programming language development GUIs in order to help developers to fix smart contracts in real-time, by detecting errors with state-of-the-art tools and providing a possible fix through ChatGPT.

References

[1] Nick Szabo. The idea of smart contracts, 1997.

[2] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. Cryptology ePrint Archive, Paper 2016/633, 2016. https://eprint.iacr.org/2016/633.

[3] Alex Graves. Generating sequences with recurrent neural networks, 2013.

[4] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019.

[5] Teo Susnjak. Chatgpt: The end of online exam integrity? *arXiv preprint arXiv:2212.09292*, 2022.

[6] Michael Dowling and Brian Lucey. Chatgpt for (finance) research: The bananarama conjecture. *Finance Res. Letters*, page 103662, 2023.

[7] Katharina Jeblick, Balthasar Schachtner, Jakob Dexl, Andreas Mittermeier, Anna Theresa Stüber, Johanna Topalis, Tobias Weber, Philipp Wesp, Bastian Sabel, Jens Ricke, et al. Chatgpt makes medicine easy to swallow: An exploratory case study on simplified radiology reports. *arXiv preprint arXiv:2212.14882*, 2022.

[8] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of chatgpt. *arXiv preprint arXiv:2301.08653*, 2023.

[9] Sarwar Sayeed, Hector Marco-Gisbert, and Tom Caira. Smart contract: Attacks and protections. *IEEE Access*, 8:24416–24427, 2020.

[10] Josselin Feist, Gustavo Grieco, and Alex Groce. Slither: A static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 8–15, May 2019.

[11] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *2019 34th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, pages 1186–1189, 2019.

[12] Rick Fontein. Comparison of static analysis tooling for smart contracts on the evm. In *28th Twente Student conference on IT*, 2018.

[13] Sergei Tikhomirov, Ekaterina Voskresenskaya, Ivan Ivanitskiy, Ramil Takhaviev, Evgeny Marchenko, and Yaroslav Alexandrov. Smartcheck: Static analysis of ethereum smart contracts. In *2018 IEEE/ACM 1st International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pages 9–16, May 2018.

[14] Petar Tsankov, Andrei Dan, Dana Drachsler-Cohen, Arthur Gervais, Florian Bünzli, and Martin Vechev. Securify: Practical security analysis of smart contracts. In *Proc. of the 2018 ACM SIGSAC Conf. on Computer and Communications Security*, CCS '18, page 67–82, New York, NY, USA, 2018. Association for Computing Machinery.

[15] Loi Luu, Duc-Hiep Chu, Hrishi Olickel, Prateek Saxena, and Aquinas Hobor. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 254–269, New York, NY, USA, 2016. Association for Computing Machinery.

[16] Mohammad Aljanabi, Mohanad Ghazi, Ahmed Hussein Ali, Saad Abas Abed, et al. Chatgpt: Open possibilities. *Iraqi Journal For Computer Science and Mathematics*, 4(1):62–64, 2023.

[17] Xiaoming Zhai. Chatgpt user experience: Implications for education. *Available at SSRN 4312418*, 2022.

[18] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. Chatgpt for good? on opportunities and challenges of large language models for education. 2023.

[19] Ömer Aydın and Enis Karaarslan. Openai chatgpt generated literature review: Digital twin in healthcare. *Available at SSRN 4308687*, 2022.

[20] Som Biswas. Chatgpt and the future of medical writing, 2023.

[21] Aidan Gilson, Conrad Safranek, Thomas Huang, Vimig Socrates, Ling Chi, Richard Andrew Taylor, and David Chartash. How well does chatgpt do when taking the medical licensing exams? the implications of large language models for medical education and knowledge assessment. *medRxiv*, pages 2022–12, 2022.

[22] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of chatgpt. *CoRR*, abs/2301.08653, 2023.

[23] Julian Aron Prenner, Hlib Babii, and Romain Robbes. Can openai's codex fix bugs? an evaluation on quixbugs. In *Proc. of the Third Int. Workshop on Automated Program Repair*, APR '22, page 69–75, New York, NY, USA, 2022. Association for Computing Machinery.

[24] Chunqiu Steven Xia and Lingming Zhang. Conversational automated program repair, 2023.

[25] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. Inferfix: End-to-end program repair with llms, 2023.

[26] Thomas Durieux, João F. Ferreira, Rui Abreu, and Pedro Cruz. Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. ACM, jun 2020.

Authorized licensed use limited to: Korea University. Downloaded on January 23,2026 at 08:37:30 UTC from IEEE Xplore. Restrictions apply.